

05.562 · Fundamentos de Computadores · 2020-21

PAC1 - Primera prueba de evaluación continuada

Presentación

Esta PEC se focaliza en los sistemas básicos de codificación de la información. Es muy importante conocer como se representa la información dentro de un computador antes de introducir los circuitos combinacionales y secuenciales. La PEC contiene un conjunto de problemas relacionados con los contenidos del Módulo 2.

Competencias

- Saber como se representa la información y, en particular, los números de forma digital: números naturales y enteros, tanto en signo y magnitud como en complemento a 2.
- Entender los mecanismos de cambios de base en la representación de números.

Objetivos

- Saber representar un mismo valor numérico en bases diferentes (2,10,16).
- Comprender los conceptos de rango y precisión de un formato de codificación de la información numérica en un computador, y también los conceptos de rebosadura y de error de representación.
- Saber representar y operar números naturales en binario.
- Saber representar y operar números enteros en signo y magnitud en base 2.
- Saber representar y operar números enteros en complemento a 2.
- Saber representar y operar números fraccionarios en coma fija.
- Conocer otros tipos de representaciones para almacenar información en un computador.

Recursos

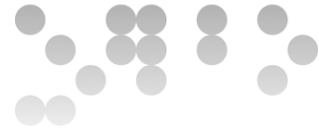
Los recursos que se recomienda usar por esta PEC son los siguientes:

Básicos: El módulo 2 de los materiales.

Complementarios: No utilizéis la calculadora para resolver los problemas puesto que en el examen no la podréis utilizar.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.



- La valoración esta indicada en cada uno de los subapartados.

Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigiós al consultor responsable de vuestra aula.
- Hay que librar la solución en un fichero PDF usando una de las plantillas libradas conjuntamente con este enunciado.
- Se debe entregar a través de la aplicación de **Entrega y registro de AC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es lo **3 de marzo** (a las 24 horas).

Descripción de la PEC a realizar

Solución de la PEC

Ejercicio 1 [15%]

Dada la secuencia de dígitos $A = 2536$ realizad los cambios de base siguientes:

Para pasar de una base b a una base b' , usamos el método basado en el TFN, el método de la división entera o el cambio de base entre b y b^n , según convenga.

- a) [5%] Asumiendo que A es un número octal, representadlo en binario.

Para pasar de base 8 a base 2 usamos el cambio entre las bases 2^3 y 2, de forma que cada dígito octal le corresponde 3 dígitos binarios. Opcionalmente, podemos sacar los 0's no significativos de la izquierda:

$$2536_{(8)} = 10101011110_{(2)}$$

- b) [5%] Asumiendo que A es un número decimal, representadlo en hexadecimal.

Para pasar de base 10 a base 16 usamos el método de la división entera:

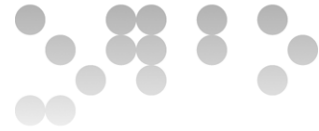
$$\begin{array}{rcl} 2536 & = & 158 \cdot 16 + 8 \\ 158 & = & 9 \cdot 16 + 14 \\ 9 & = & 0 \cdot 16 + 9 \end{array}$$

$$2536_{(10)} = 9E8_{(16)}$$

- c) [5%] Asumiendo que A es un número hexadecimal, representadlo en decimal.

Para pasar de base 16 a base 10 usamos el método basado en el TFN:

$$2536_{(16)} = 2 \cdot 16^3 + 5 \cdot 16^2 + 3 \cdot 16^1 + 6 \cdot 16^0 = 9526_{(10)}$$



Ejercicio 2 [15%]

Representad el número entero decimal -391 en las codificaciones binarias siguientes:

- a) [5%] En signo y magnitud (SM2) con 10 bits.

Usando el método de la división entera por 2, ya usado en el ejercicio anterior, encontramos que la magnitud $391_{(10)}$ se representa en binario como $110000111_{(2)}$. Esta magnitud se representa ya en 9 bits. Hay que añadir el bit de signo a la izquierda y ya tenemos la representación pedida en SM2 y 10 bits.

Así pues, la representación será $-391_{(10)} = 1110000111_{(SM2)}$

- b) [5%] En complemento a 2 (Ca2) con 10 bits.

Partimos de la representación del número en positivo y después le aplicamos la operación de cambio de signo.

El número en positivo, $391_{(10)}$, en binario y 10 bits es $0110000111_{(2)}$. Ahora le aplicamos el cambio de signo haciendo la operación de complemento a 2. Se puede calcular, por ejemplo, calculando el complemento a 1 y sumando 1 al resultado ($1001111000_{(2)} + 1 = 1001111001_{(2)}$).

Así pues, la representación será $-391_{(10)} = 1001111001_{(Ca2)}$

- c) [5%] En exceso a 512 con 10 bits.

Para representar en exceso hay que sumar al valor la magnitud del exceso y después codificar en binario el resultado obtenido: $-391 + 512 = 121$. La representación binaria es $121_{(10)} = 1111001_{(2)}$. Si hace falta, se añaden ceros no significativos a la izquierda del número binario.

Así pues, la representación será $-391_{(10)} = 0001111001_{(exceso)}$

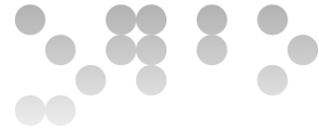
Ejercicio 3 [40%]

Dados los números binarios siguientes:

$$A = 001101010101$$

$$B = 101010101100$$

Realizad las operaciones pedidas según el formato indicado en cada apartado. Mostrad los resultados únicamente en binario (no hay que hacer ninguna conversión a decimal) usando 12 bits, e indicad y justificad en cada caso si se produce desbordamiento.



a) [10%] $A + B$ considerando que los números están codificados en signo y magnitud.

- Para sumar dos números codificados en signo y magnitud que tienen signo diferente, procedemos de la manera siguiente:

- Al tener signo diferente, se resta la magnitud más pequeña de la magnitud mayor. En nuestro caso, la magnitud de A es mayor que la de B , por lo tanto, B se resta de A .
- El resultado tendrá el signo del operando que tenga la magnitud mayor. En nuestro caso, el signo será el de A , que es positivo.
- Restamos la magnitud pequeña de la grande:

$$\begin{array}{r}
 0\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1 \leftarrow A \leftarrow \\
 \quad \quad \quad 1\quad 1\quad 1 \quad \text{acarreo} \\
 -\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 0 \leftarrow B \\
 \hline
 0\ 0\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1
 \end{array}$$

- Para obtener el resultado tenemos que añadir el signo al resultado del resto: **000010101001**_(SM2)

- No hay desbordamiento**, puesto que se suman dos números de diferente signo.

b) [10%] $A - B$ considerando que los números están codificados en signo y magnitud.

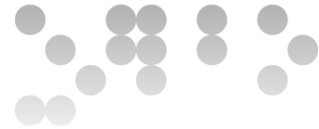
- En este caso el primer operando es positivo y el segundo operando es negativo. La operación de resta se convierte en una operación de suma de las magnitudes y añadimos al resultado el signo positivo.

- Sumamos las magnitudes de los operandos:

$$\begin{array}{r}
 \quad \quad \quad 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1 \quad \leftarrow \text{acarreo} \\
 0\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1 \leftarrow A \\
 +\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 0 \leftarrow B \\
 \hline
 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1
 \end{array}$$

- Para obtener el resultado, tenemos que añadir el signo negativo al resultado de la suma: **011000000001**_(SM2)

- No hay desbordamiento**, puesto que no hay acarreo en la última etapa de la suma de las magnitudes.



c) [10%] $A + B$ considerando que los números están codificados en complemento a 2.

- Hacemos la suma de las dos cantidades:

$$\begin{array}{r}
 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1 \quad \leftarrow \text{acarreo} \\
 0\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1 \quad \leftarrow A \\
 +\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 0 \quad \leftarrow B \\
 \hline
 1\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1
 \end{array}$$

- El resultado es: $111000000001_{(Ca2)}$
- No hay desbordamiento, puesto que se suman dos números de signo diferente.

d) [10%] $A - B$ considerando que los números están codificados en complemento a 2.

- Para restar en Ca2, convertimos la operación $A - B$ en $A + (-B)$, cambiando el signo del sustraendo, es decir, complementándolo bit a bit y sumando 1 al resultado:

$$-B = 010101010011 + 1 = 010101010100$$

$$\begin{array}{r}
 1\ 1\ 1\ 1\ 1\ 1\ 1 \quad \leftarrow \text{acarreo} \\
 0\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1 \quad \leftarrow A \\
 +\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 0 \quad \leftarrow B \\
 \hline
 1\ 0\ 0\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1
 \end{array}$$

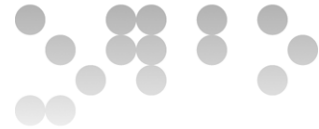
- El resultado es: $100010101001_{(Ca2)}$
- Hay desbordamiento, puesto que sumamos dos números positivos y el signo del resultado es negativo.

Ejercicio 4 [30%]

a) [10%] Dado el número real $N = 56,66$, representadlo en el formato de coma fija sin signo, con 6 bits para la parte entera y 4 bits para la parte fraccionaria. El método de aproximación que hay que aplicar es el redondeo.

Pasamos la parte entera a binario, aplicando el método de la división entera. Encontramos que $56_{(10)}$ se representa en binario como $111000_{(2)}$.

Para la parte fraccionaria aplicamos el método correspondiente, calculando 5 bits para poder redondear:



$$\begin{array}{rcl}
 0,66 & \cdot 2 = & 1,32 \\
 0,32 & \cdot 2 = & 0,64 \\
 0,64 & \cdot 2 = & 1,28 \\
 0,28 & \cdot 2 = & 0,56 \\
 0,56 & \cdot 2 = & 1,12
 \end{array}$$



Como que el quinto bit es 1 hay que incrementar los 4 bits de la parte fraccionaria.

$$0,66_{(10)} = 0,1011_{(2)}$$

Juntamos la parte entera y fraccionaria: **111000,1011**₍₂₎

- b) [10%] Indica en decimal el número mayor que se puede representar en el formato anterior de coma fija.

El número mayor será aquel que tiene todos los bits de la representación a 1. Es decir, sería 111111,1111₍₂₎. Este número es $1000000_{(2)} - 0,0001_{(2)} = 64 - 0,0625 = \mathbf{63,9375}$.

- c) [10%] Dado el número real $N = 56,66$, representadlo en coma flotante según el formato siguiente:

S	Exponente	Mantisa
1bit	4 bits	8 bits

en el cual:

- El bit de signo, S, vale 0 para cantidades positivas y 1 para negativas.
- El exponente se representa a exceso a 8.
- Hay bit implícito.
- La mantisa está normalizada en la forma 1,X.
- El método de aproximación que hay que aplicar es el truncamiento.

Del apartado anterior tenemos la representación binaria del número. Teniendo en cuenta el formato mencionado hay que tener calculados 8 dígitos binarios a partir del primer dígito significativo. Es decir:

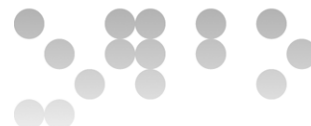
$$56,66 = 111000,101_{(2)}$$

Tenemos que normalizar la mantisa, desplazando la coma 5 posiciones a la izquierda:

$$56,66 = 1,11000101_{(2)} \cdot 2^5$$

Identificamos cada campo:

- Signo: Positivo, S=0.
- Exponente: 5. Hay que representarlo en exceso. Por lo tanto, tenemos que sumarle el exceso, $8 + 5 = 13$, que en base 2 es 1101₍₂₎.



- Mantisa: $1,11000101_{(2)}$. Como que lo tenemos que representar con bit implícito eliminamos el 1 de la parte entera. Así pues, la mantisa será $11000101_{(2)}$.

El número en el formato solicitado es:

0	1101	11000101
---	------	----------